

# ACL em arquivos Linux

Permissões finas no sistema de arquivos: como usar POSIX ACLs para controlar acesso por usuário e grupo além das permissões tradicionais.

## Introdução

Access Control Lists (ACLs) estendem o modelo clássico de permissões Unix (owner/group/other) permitindo atribuir permissões específicas a vários usuários e grupos sobre arquivos e diretórios. Esse mecanismo é útil quando o modelo de três campos não atende às regras de acesso da aplicação ou do fluxo de trabalho.

Do ponto de vista teórico, ACLs representam uma forma prática de implementar uma matriz de acesso (access matrix) no nível do sistema de arquivos: linhas são sujeitos (usuários/grupos) e colunas são objetos (arquivos/dirs), cada célula contém as operações permitidas.

## Conceitos essenciais

- **Entradas básicas:** `user::` (permissão do dono), `group::` , `other::` .
- **Entradas nomeadas:** `user:giu:rwX` OU `group:devs:rX` — permissões para usuários/grupos específicos.
- **Mask (máscara):** limita as permissões efetivas para entradas de grupo e entradas nomeadas. O valor do mask é aplicado como um "AND" entre a permissão pedida e o mask.
- **Default ACLs:** prefixadas por `d:` , aplicam permissões herdadas a novos arquivos e subdiretórios criados dentro de um diretório.
- **Ferramentas:** `getfacl` (ler), `setfacl` (modificar).

## Casos de uso práticos

- Pastas colaborativas onde vários usuários precisam de permissões distintas sem formar muitos grupos.
- Ambientes multi-tenant (hospedagem, contêineres) em que cada cliente precisa de regras específicas de acesso.
- Diretórios de projeto com regras diferentes para desenvolvedores, revisores e operadores de deploy.

Arquitetura simples: sistema de arquivos com ACLs + serviço de autenticação (LDAP/SSSD) para mapear identidades; ACLs resolvem "quem pode fazer o quê" no disco.

## Comandos básicos

Instalar o pacote para ACL

```
sudo apt update && sudo apt install acl -y
```

Listar ACLs:

```
getfacl arquivo_ou_diretorio
getfacl -R diretorio    # recursivo
```

Adicionar / modificar entradas:

```
setfacl -m u:giu:rw- arquivo      # dá rw para giu
setfacl -m g:devs:r-x arquivo     # dá rx para o grupo devs
setfacl -m m::rwx arquivo         # ajusta a mask
```

Entradas default (para diretórios):

```
setfacl -m d:u:giu:rwx diretorio  # todas as criações herdarão esta entrada
setfacl -R -d -m g:devs:rwx diretorio # aplica default recursivamente
```

Remover entradas / limpar ACLs:

```
setfacl -x u:giu arquivo      # remove entrada de giu
setfacl -b arquivo            # remove todas as ACLs (backup: cuidado!)
```

Utilizando o `ls -l` para checar a presença de ACL. Se na permissão conter um `+`, ACL é definida para este arquivo!

```
-rwxr-xr-x  1 urubu200 urubu200 1242 Aug 12 20:39 monitorament_v1.sh  # este arquivo não tem ACL de
-rw-r--r--+ 1 urubu200 urubu200 1646 Aug 17 10:27 registro.log        # este arquivo TEM ACL defini
```

```
getfacl registro.log
# file: registro.log
# owner: urubu200
# group: urubu200
user::rw-
user:urubu200:r--
group::r--
mask::r--
other::r--
```

Perceba a presença da `mask`, que determina as permissões efetivas para entradas nomeadas e grupos. A máscara ACL define o conjunto máximo de permissões ACL, independentemente de existirem permissões que excedam essa máscara. A máscara ACL é atualizada sempre que você executa o comando `setfacl`, a menos que especifique, por meio da flag `-n`, que não deseja atualizá-la.

```
setfacl -n -m u:giu:rwx arquivo_ou_diretorio
```

Nota: se o seu conjunto máximo de permissões diferir da entrada de máscara, será exibida uma linha efetiva que calcula o conjunto "real" de entradas de ACL utilizadas.

```
sudo setfacl -n -m u:urubu200:rw- registro_recursos.log
getfacl registro_recursos.log
# file: registro_recursos.log
# owner: root
# group: root
user::r--
user:urubu200:rw-          #effective:---
group::r--                 #effective:---
mask:---
```

### Criação de listas de controle de acesso padrão em diretórios

Você tem a opção de criar listas de controle de acesso padrão em diretórios. As listas de controle de acesso padrão são usadas para criar entradas de ACL em um diretório que serão herdadas por objetos contidos nele, como arquivos ou subdiretórios.

- Os arquivos criados neste diretório herdam as entradas de ACL especificadas no diretório pai.
- Os subdiretórios criados neste diretório herdam as entradas de ACL, bem como as entradas de ACL padrão, do diretório pai
- Para criar entradas de ACL padrão, especifique a opção `-d` ao definir a ACL usando o comando `setfacl`.

Por exemplo, para atribuir permissões de leitura a todos os arquivos criados em um diretório, você executaria o seguinte comando:

```
setfacl -d -m u::r nome_do_diretório
```

Se você quiser travar apenas o usuário com permissão de leitura (o famoso chmod 400)

```
setfacl -d -m u::r,g::- ,o::- nome_do_diretório
setfacl -R -d -m u::r,g::- ,o::- nome_do_diretório      # de forma recursiva
```

E para deletar o ACL padrão no diretório

```
setfacl -kR nome_do_diretório
```

### Prós e Contras

| Prós                                                            | Contras / limitações                                                                       |
|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Permite regras finas por usuário/grupo sem criar muitos grupos. | Introduz complexidade: mais itens para auditar e manter.                                   |
| Suporta herança via default ACLs para diretórios.               | Nem todo FS/mount tem ACLs habilitadas por padrão em todos os ambientes, verificar sempre. |

| Prós                                                      | Contras / limitações                                                                                       |
|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Compatível com as ferramentas usuais (getfacl / setfacl). | Máscara (mask) pode confundir se não for entendida; operações com <code>chmod</code> podem alterar a mask. |

Quando **NÃO** usar: para isolamento forte entre domínios de confiança, prefira mecanismos de controle de acesso obrigatórios ([SELinux](#), [AppArmor](#)) ou separar em storage distintos; também considere gestão por grupos se a política for simples.

### Exercício resolvido — Caso prático

Contexto: você administra um servidor de arquivos e precisa preparar a área do projeto `/projetoAraripe` com as seguintes regras:

1. O grupo `devs` deve ter acesso total (rwx) à pasta e a todos os arquivos e subpastas.
2. A **giu** precisa de acesso total (rwx) a todos os arquivos do projeto, incluindo arquivos que serão criados no futuro.
3. A **fernanda** precisa apenas de leitura em todo o projeto, mas terá permissão de escrita apenas na subpasta `docs`.
4. Outros usuários não devem ter acesso.

Execute os comandos no servidor (como `root` ou usando `sudo`).

### Tarefas

Implemente a política acima usando ACLs.

### Dicas

- Use `chown` e `chmod` para definir proprietários e bits iniciais.
- Use `setfacl -m` para entradas imediatas e `-d -m` para defaults herdáveis.
- Verifique com `getfacl -R /projetoAraripe` e `ls -l`.

### Solução (exemplo)

Exemplo sequencial de comandos que implementam a política:

```
# criar diretório (se necessário)
sudo mkdir -p /projetoAraripe

# definir dono e grupo do projeto
sudo chown root:devs /projetoAraripe

# aplicar SGID no diretório para herdar grupo em novos arquivos (opcional, mas útil)
sudo chmod 2770 /projetoAraripe

# dar acesso full ao grupo devs recursivamente
sudo setfacl -R -m g:devs:rwx /projetoAraripe
sudo setfacl -R -d -m g:devs:rwx /projetoAraripe
```

```
# giu: acesso completo agora e em criações futuras
sudo setfacl -R -m u:giu:rwX /projetoAraripe
sudo setfacl -R -d -m u:giu:rwX /projetoAraripe

# fernanda: leitura em todo o projeto
sudo setfacl -R -m u:fernanda:r-X /projetoAraripe
sudo setfacl -R -d -m u:fernanda:r-X /projetoAraripe

# permitir escrita para fernanda somente na subpasta docs
sudo mkdir -p /projetoAraripe/docs
sudo setfacl -R -m u:fernanda:rwX /projetoAraripe/docs
sudo setfacl -R -d -m u:fernanda:rwX /projetoAraripe/docs

# garantir máscara ampla para que entradas nomeadas funcionem conforme esperado
sudo setfacl -R -m m::rwX /projetoAraripe

# verificação
getfacl -R /projetoAraripe
ls -l /projetoAraripe
```

Observação: o comando `chmod 2770` aplica o bit SGID (2), garantindo que novos arquivos criados herdem o grupo `devs` . A combinação de SGID + default ACLs costuma ser prática em ambientes colaborativos.

## Checagens e solução de problemas

- Se `getfacl` retornar erros, verifique se o sistema de arquivos suporta ACLs. Em muitos sistemas ext4 é ativado por padrão; caso necessário, remonte com:

```
sudo mount -o remount,acl /ponto_de_montagem
```

- Lembre-se que a `mask` pode restringir as permissões efetivas: se um usuário aparece com permissão mas não consegue, verifique o valor do `mask` em `getfacl` .
- O uso de `chmod` pode alterar a mask; após mudanças complexas, verifique novamente as ACLs.

## Entrega case projeto

Agora que você entende um pouco mais de permissionamento de arquivos em servidores Linux, para o projeto de cada um desenvolver

1. Tabela com os usuários e suas permissões
2. Diretórios que serão desenvolvidos para o projeto, permissões desses diretórios
3. Componentes de software que serão provisionados (app NodeJS, Banco MySQL, Java, etc.)
4. Listar as permissões de cada usuário perante cada componente de software
5. Para as instâncias configuradas na AWS EC2 listar quais portas serão necessárias e de quais ranges de IP serão liberadas

Subir esse documento em .pdf no Moodle. Entrega 1 por grupo de pesquisa e inovação

### Links úteis

- `man getfacl` / `man setfacl`
- Documentação [POSIX ACLs](#)
- [Mastering Linux Access Control Lists \(ACLs\): A Hands-on guide](#)
- [Docs Oracle: Using Access Control Lists \(ACLs\)](#)
- [Red Hat: An introduction to Linux Access Control Lists \(ACLs\)](#)
- [What is Discretionary Access Control \(DAC\)?](#)
- [IBM: O que é gerenciamento de acesso e identidade \(IAM\)?](#)

---

Execute comandos com cuidado em **ambientes de produção**.